

RecoverAI — Root-Cause Revenue Recovery Agent

Razorpay AI Buildathon — Track 03: AI Revenue Recovery

Build Window	21 August 2026 → 2 September 2026 (13 days)
Tool Stack	Antigravity (agentic IDE) + Razorpay Test-Mode APIs + razorpay-mcp-server
Track Fit	AI Revenue Recovery — detect, diagnose, decide, recover, audit
Deliverables	Public GitHub repo, 5-min pitch deck, architecture diagram, live demo

One-line pitch

Most failed-payment bots retry blindly. RecoverAI diagnoses **why** revenue is at risk (insufficient funds, expired mandate, checkout abandonment, overdue B2B invoice) before choosing a bounded, compliant recovery action — then proves it with a measured recovery report and a full audit trail across a test-mode transaction batch.

1. Project Overview

Problem: Revenue leaks through multiple channels — payment failures, checkout abandonment, failed subscriptions, overdue receivables. Each needs a different fix, not one generic retry.

Solution: An agentic pipeline with five stages — Detect → Diagnose → Decide → Execute → Measure/Audit — built on Razorpay's test-mode Payments, Subscriptions, Payment Links and Invoices APIs, driven through the razorpay-mcp-server so an LLM agent can call real endpoints safely.

Core modules

- **Webhook Listener:** ingests payment.failed, subscription.halted, order.created-without-paid, invoice overdue events
- **Root-Cause Classifier:** buckets failures (insufficient funds / bank decline / expired card / mandate failure / OTP drop-off / price friction)
- **Policy Engine:** maps root cause → bounded intervention, enforces stopping rules (max retries, cool-off windows, discount caps, DND compliance)
- **Execution Layer:** creates retries / payment links / reminders via Razorpay APIs, sends via email or WhatsApp sandbox
- **Recovery Ledger + Dashboard:** logs every case, shows ■ at risk vs ■ recovered, recovery rate by bucket, full audit trail

2. Day-by-Day Plan (21 Aug → 2 Sep)

13-day build broken into 4 phases: Setup & Design, Core Agent Build, Integration & Metrics, Polish & Submission. Each day has a clear, checkable output so progress is measurable.

Day	Date	Phase	Tasks	Checkpoint / Output
Day 1	Fri 21 Aug	Setup	Finalize scope & name (RecoverAI). Create Razorpay test account, generate API keys. Set up Antigravity project workspace. Clone/review razorpay-mcp-server repo.	Repo initialized, Razorpay test keys working, MCP server running locally
Day 2	Sat 22 Aug	Setup	Design system architecture diagram (5-stage pipeline). Define data schema for transactions, failure events, recovery cases. Define stopping-rule config format (JSON).	Architecture diagram + schema docs committed
Day 3	Sun 23 Aug	Setup	Build synthetic test-data generator: simulate failed payments, abandoned checkouts, overdue invoices via Razorpay test mode + scripted mock events.	Script that produces a repeatable 'batch' of ~50-100 test-mode revenue-risk events
Day 4	Mon 24 Aug	Core Build	Build Webhook Listener service (Flask/FastAPI) to capture payment.failed, subscription.halted, invoice.overdue events and store them in the Recovery Ledger DB.	Listener captures and logs all event types correctly
Day 5	Tue 25 Aug	Core Build	Build Root-Cause Classifier: rule-based + LLM-assisted classification of each failure into a bucket, using Razorpay error codes + payment metadata + history.	Classifier tagging each event with a root-cause bucket + confidence
Day 6	Wed 26 Aug	Core Build	Build Policy Engine: map each root-cause bucket to an intervention type; implement stopping rules (max attempts, cool-off, discount cap, DND check).	Policy engine returns a bounded action + reason for any input event
Day 7	Thu 27 Aug	Core Build	Build Execution Layer: wire policy decisions to real Razorpay test-mode calls (create Payment Link, retry subscription charge, generate invoice reminder).	End-to-end: event in → classified → policy → real API action executed
Day 8	Fri 28 Aug	Integration	Add notification channel (email via SMTP sandbox, or WhatsApp Business sandbox) to actually deliver the recovery nudge/payment link to the customer.	Customer-facing message sent with working payment link
Day 9	Sat 29 Aug	Integration	Build Promise-to-Pay tracker for B2B receivables + Mandate Retry Sequencer logic (bounded retry schedule for failed autopay/mandates).	Both flows working on sample overdue invoice + failed mandate cases
Day 10	Sun 30 Aug	Integration	Build Recovery Dashboard: revenue at risk vs recovered, recovery rate by bucket, cost of intervention, per-case audit trail viewer. Run full batch test.	Dashboard renders live metrics from a full batch run

Day	Date	Phase	Tasks	Checkpoint / Output
Day 11	Mon 31 Aug	Metrics & Polish	Run 2-3 full batch simulations, tune classifier + policy thresholds, fix edge cases (duplicate events, race conditions, failed sends), write unit tests.	Stable batch run with clean, believable recovered-revenue numbers
Day 12	Tue 1 Sep	Submission Prep	Write README (problem, architecture, how to run), record demo video/GIF, prepare 5-min pitch deck (problem, architecture, demo, metrics, what's next).	README complete, pitch deck drafted, demo recording ready
Day 13	Wed 2 Sep	Submission	Final rehearsal of 5-min pitch, polish architecture diagram, push final commit, make repo public, submit application before deadline.	GitHub repo public + application submitted

3. How to Run This With Antigravity

Antigravity works best when you give it one clear module target per session and let it iterate with tests. Suggested prompting flow, mapped to the days above:

- **Session 1 (Days 1-2):** Ask Antigravity to scaffold the repo structure, README skeleton, and the architecture diagram description before writing code.
- **Session 2 (Day 3):** Ask it to generate a synthetic-data script that simulates realistic Razorpay test-mode failure/abandonment events, with a fixed random seed for reproducibility.
- **Session 3 (Days 4-7):** Build one module per prompt — webhook listener, then classifier, then policy engine, then execution layer. Have Antigravity write tests alongside each module so regressions are caught immediately.
- **Session 4 (Days 8-10):** Wire the notification channel and dashboard; ask Antigravity to run the full batch pipeline end-to-end and print a summary report.
- **Session 5 (Days 11-13):** Use Antigravity for polish — refactor, write the README, generate the pitch deck outline, and do a final lint/test pass before submission.

4. Panel Evaluation Checklist

- Does the demo show a **batch** of transactions, not just one cherry-picked case?
- Is **revenue recovered** clearly measured and shown, alongside recovery rate by root-cause bucket?
- Are **stopping rules** (max attempts, cool-off, discount caps, DND/consent compliance) visible in the demo or code?
- Is there a **per-case audit trail** — detection → diagnosis → decision → action → outcome?
- Does the root-cause classification step add real value over a naive blind-retry bot?
- Is the code built on real Razorpay test-mode API calls (not fully mocked)?

Prepared as a working plan for the Razorpay AI Buildathon — Track 03: AI Revenue Recovery. Adjust dates/tasks as needed based on actual build velocity.